RSA and Shor's Algorithm

From Number Theory to Quantum Factoring

QC-Course 2026 — lesson notes

companion to `IBM/16_Shor_Period_Finding_to_RSA.ipynb` and `IQM/16_Shor_Period_Finding_to_RSA.ipynb`

October 23, 2026

Abstract

These notes explain, from first principles, how *asymmetric* (public-key) cryptography works, why RSA in particular is secure against classical computers, and how a quantum computer running *Shor's algorithm* breaks it. We state and explain every number-theoretic theorem RSA relies on (Fermat, Euler, Bézout, the Chinese Remainder Theorem), prove that RSA decryption inverts encryption, and then build Shor's algorithm step by step: the reduction from factoring to order finding, quantum phase estimation of the modular-multiplication operator, and continued-fraction post-processing. Practical "recipes" and fully worked numerical examples accompany each step, and map directly onto the cells of the companion Jupyter notebook.

Contents

1 Symmetric vs. asymmetric cryptography **1**

2 Number-theoretic background **2**
- 2.1 gcd, Bézout, and modular inverses . . . 2
- 2.2 Euler's totient function . . . 2
- 2.3 Fermat's little theorem and Euler's theorem . . . 2
- 2.4 The Chinese Remainder Theorem . . . 3

3 The RSA cryptosystem **3**
- 3.1 Key generation, encryption, decryption . . . 3
- 3.2 Why decryption works . . . 3
- 3.3 A fully worked (small) example . . . 4
- 3.4 Security: what an attacker must do . . . 4

4 Shor's algorithm **5**
- 4.1 Step 1 — Reduce factoring to order finding . . . 5
- 4.2 Step 2 — Compute the order on a quantum computer . . . 5
- 4.3 Step 3 — Recover r by continued fractions . . . 6
- 4.4 The complete recipe . . . 6
- 4.5 Worked quantum examples . . . 7
- 4.6 The bottleneck: modular exponentiation . . . 7
- 4.7 Reality check and post-quantum cryptography . . . 7

5 From the lesson to the notebook **8**

Exercises 8

References 8

1 Symmetric vs. asymmetric cryptography

In a *symmetric* cipher (AES, ChaCha20, ...) the same secret key k both encrypts and decrypts: $c = E_k(m)$, $m = D_k(c)$. This is fast, but it has a famous weakness — the *key-distribution problem*: before Alice and Bob can talk securely, they must already share k over some secure channel. For n mutually communicating parties one needs $\binom{n}{2}$ shared keys.

Asymmetric (public-key) cryptography, introduced by Diffie and Hellman (1976) and first realised concretely by Rivest, Shamir and Adleman (1977), removes this problem. Each party publishes a **public key** pk and keeps a **private key** sk. Anyone can encrypt *to* Bob using his public key, but only Bob — holder of the matching private key — can decrypt:

$$c = E_{\mathrm{pk}}(m), \qquad m = D_{\mathrm{sk}}(c).$$

Definition 1.1 (One-way and trapdoor functions)**.** A function f is *one-way* if $y = f(x)$ is easy to compute but $x = f^{-1}(y)$ is computationally infeasible. It is a *trapdoor* one-way function if there is extra secret information (the trapdoor) that makes inverting f easy again.

Public-key cryptography needs trapdoor one-way functions. For RSA the trapdoor is the **factorisation of the modulus**: multiplying two large primes p, q to get $N = pq$ is easy, but recovering p, q from N is (classically) believed hard — unless you know the trapdoor. Shor's algorithm destroys this asymmetry on a quantum computer.

2 Number-theoretic background

All of RSA lives in *modular arithmetic*. We write $a \equiv b \pmod{n}$ to mean $n \mid (a - b)$, i.e. a and b leave the same remainder on division by n. The set of residues $\mathbb{Z}/n\mathbb{Z} = \{0, 1, \ldots, n-1\}$ is a commutative ring under $+$ and $\times$ modulo n.

2.1 gcd, Bézout, and modular inverses

Theorem 2.1 (Bézout's identity)**.** *For integers a, b not both zero, there exist integers x, y with*

$$ax + by = \gcd(a, b).$$

Moreover x, y are computed in $O(\log \min(a, b))$ steps by the extended Euclidean algorithm.

Idea. The set $S = \{ax + by : x, y \in \mathbb{Z}\} \cap \mathbb{N}_{>0}$ is non-empty; let $d = ax_0 + by_0$ be its least element. Dividing a by d with remainder, $a = qd + r$ with $0 \le r < d$, gives $r = a - qd = a(1 - qx_0) - b(qy_0) \in S \cup \{0\}$. Minimality of d forces $r = 0$, so $d \mid a$ and likewise $d \mid b$; hence $d \mid \gcd(a, b)$. Conversely $\gcd(a, b) \mid d$ because it divides every term $ax + by$. Thus $d = \gcd(a, b)$. □

Corollary 2.2 (Existence of modular inverses)**.** *An element $a \in \mathbb{Z}/n\mathbb{Z}$ has a multiplicative inverse (an a^{-1} with $a\,a^{-1} \equiv 1 \pmod{n}$) **iff** $\gcd(a, n) = 1$. The inverse is the coefficient x in Bézout's identity $ax + ny = 1$, reduced modulo n.*

This corollary is exactly what lets us turn the public exponent e into the private exponent d in RSA: $d \equiv e^{-1} \pmod{\varphi(N)}$.

Definition 2.3 (Group of units)**.** The *units* of $\mathbb{Z}/n\mathbb{Z}$ are the invertible residues, $(\mathbb{Z}/n\mathbb{Z})^* = \{\, a : 1 \le a \le n,\ \gcd(a,n) = 1 \,\}$. This is an abelian group under multiplication modulo n.

2.2 Euler's totient function

Definition 2.4 (Euler's totient)**.** $\varphi(n) = \left|(\mathbb{Z}/n\mathbb{Z})^*\right|$ is the number of integers in $\{1, \dots, n\}$ coprime to n.

Proposition 2.5 (Totient of a product of distinct primes)**.** *If $p \neq q$ are primes then $\varphi(pq) = (p-1)(q-1)$. More generally φ is* multiplicative*: $\varphi(mn) = \varphi(m)\varphi(n)$ whenever $\gcd(m,n) = 1$, and $\varphi(p) = p - 1$ for a prime p.*

Sketch. Among $\{1, \dots, pq\}$ the numbers *not* coprime to pq are the q multiples of p and the p multiples of q, with the single number pq counted twice. Hence $\varphi(pq) = pq - p - q + 1 = (p-1)(q-1)$. Multiplicativity follows from the Chinese Remainder Theorem below, which gives a ring isomorphism $\mathbb{Z}/mn\mathbb{Z} \cong \mathbb{Z}/m\mathbb{Z} \times \mathbb{Z}/n\mathbb{Z}$ and hence a bijection of unit groups. □

2.3 Fermat's little theorem and Euler's theorem

Theorem 2.6 (Fermat's little theorem)**.** *If p is prime and $p \nmid a$, then $a^{p-1} \equiv 1 \pmod{p}$. Equivalently, $a^p \equiv a \pmod{p}$ for* every *integer a.*

Theorem 2.7 (Euler's theorem)**.** *If $\gcd(a,n) = 1$ then*

$$a^{\varphi(n)} \equiv 1 \pmod{n}.$$

Proof. Work in the finite abelian group $(\mathbb{Z}/n\mathbb{Z})^*$ of order $\varphi(n)$. By Lagrange's theorem the order of any element divides the order of the group, so $a^{\varphi(n)} = 1$ in $(\mathbb{Z}/n\mathbb{Z})^*$, i.e. $a^{\varphi(n)} \equiv 1 \pmod{n}$. (Fermat's little theorem is the special case $n = p$ prime, where $\varphi(p) = p - 1$.) □

Euler's theorem is the engine of RSA: raising to the power $\varphi(N)$ acts like the identity on units, so exponents can be reduced modulo $\varphi(N)$.

2.4 The Chinese Remainder Theorem

Theorem 2.8 (CRT)**.** *If $\gcd(m,n) = 1$, the system $x \equiv a \pmod{m}$, $x \equiv b \pmod{n}$ has a unique solution modulo mn. Equivalently, the map*

$$\mathbb{Z}/mn\mathbb{Z} \longrightarrow \mathbb{Z}/m\mathbb{Z} \times \mathbb{Z}/n\mathbb{Z}, \qquad x \longmapsto (x \bmod m,\ x \bmod n)$$

is a ring isomorphism.

CRT is used both to *prove* RSA correct (Section 3.2) and to *speed it up* in practice (decrypting modulo p and q separately is $\sim 4\times$ faster than modulo N).

3 The RSA cryptosystem

3.1 Key generation, encryption, decryption

Recipe R1 — RSA key generation

1. Choose two large distinct primes p, q (e.g. 1024 bits each for a 2048-bit key).
2. Compute the modulus $N = pq$ and the totient $\varphi(N) = (p-1)(q-1)$.
3. Choose a public exponent e with $1 < e < \varphi(N)$ and $\gcd(e, \varphi(N)) = 1$ (commonly $e = 65537$).
4. Compute the private exponent $d \equiv e^{-1} \pmod{\varphi(N)}$ via the extended Euclidean algorithm (Corollary 2.2).
5. **Public key** $\mathrm{pk} = (N, e)$. **Private key** $\mathrm{sk} = (N, d)$.
   Discard (or guard) $p, q, \varphi(N)$ — knowing any of them breaks the scheme.

For a message represented as an integer m with $0 \le m < N$:

$$\text{(encrypt)} \quad c = m^e \bmod N, \qquad\qquad \text{(decrypt)} \quad m = c^d \bmod N.$$

3.2 Why decryption works

Theorem 3.1 (RSA correctness)**.** *With the parameters of Recipe R1, for every integer m with $0 \le m < N$,*

$$(m^e)^d \equiv m \pmod{N}.$$

Proof. By construction $ed \equiv 1 \pmod{\varphi(N)}$, so $ed = 1 + k\varphi(N)$ for some integer $k \ge 0$, where $\varphi(N) = (p-1)(q-1)$. We show $m^{ed} \equiv m$ modulo each prime and combine with CRT (Theorem 2.8).

Modulo p. If $p \mid m$ then $m^{ed} \equiv 0 \equiv m \pmod{p}$. Otherwise $\gcd(m, p) = 1$, and Fermat's little theorem (Theorem 2.6) gives $m^{p-1} \equiv 1 \pmod{p}$. Since $(p-1) \mid \varphi(N)$,

$$m^{ed} = m^{1+k\varphi(N)} = m \cdot \left(m^{p-1}\right)^{k(q-1)} \equiv m \cdot 1^{\,k(q-1)} = m \pmod{p}.$$

Modulo q. The same argument gives $m^{ed} \equiv m \pmod{q}$.

Since $p \neq q$ are coprime and $m^{ed} - m$ is divisible by both p and q, it is divisible by $N = pq$; hence $m^{ed} \equiv m \pmod{N}$. □

3.3 A fully worked (small) example

Example 3.2 (RSA by hand)**.** Take $p = 5$, $q = 11$. Then

$$N = 55, \qquad \varphi(N) = (5-1)(11-1) = 40.$$

Choose $e = 3$ (indeed $\gcd(3, 40) = 1$). The private exponent solves $3d \equiv 1 \pmod{40}$; since $3 \cdot 27 = 81 = 2 \cdot 40 + 1$, we get $d = 27$.

$$\mathrm{pk} = (55, 3), \qquad \mathrm{sk} = (55, 27).$$

Encrypt $m = 7$:

$$c = 7^3 \bmod 55 = 343 \bmod 55 = 13.$$

Decrypt $c = 13$ (using CRT to keep the numbers small):

$$13 \equiv 3 \pmod{5}, \quad 3^{27} \equiv 3^{27 \bmod 4} = 3^3 = 27 \equiv 2 \pmod{5};$$

$$13 \equiv 2 \pmod{11}, \quad 2^{27} \equiv 2^{27 \bmod 10} = 2^7 = 128 \equiv 7 \pmod{11}.$$

CRT reassembles $x \equiv 2 \pmod 5$, $x \equiv 7 \pmod{11}$ into $x = 7$. Decryption returns the original $m = 7$. ✓

Remark 3.3 (The textbook "big" example). The classic RSA example (and the one in `13_RSA.ipynb`) uses $p = 61$, $q = 53$, so $N = 3233$, $\varphi(N) = 3120$, $e = 17$, $d = 2753$. Encrypting $m = 65$ gives $c = 65^{17} \bmod 3233 = 2790$, and $2790^{2753} \bmod 3233 = 65$ again.

3.4 Security: what an attacker must do

Proposition 3.4 (Knowing $\varphi(N)$ is as hard as factoring)**.** *Given $N = pq$, recovering $\varphi(N) = (p-1)(q-1)$ is computationally equivalent to factoring N.*

Proof. If p, q are known, $\varphi(N)$ is immediate. Conversely, from $N = pq$ and $\varphi(N) = N - (p+q) + 1$ we recover $p + q = N - \varphi(N) + 1 =: s$. Then p, q are the roots of $t^2 - st + N = 0$, i.e. $p, q = \frac{1}{2}\big(s \pm \sqrt{s^2 - 4N}\big)$ — a constant-time computation. □

Thus the entire security of RSA rests on the presumed hardness of the **integer-factorisation problem** (and the slightly weaker "RSA problem": computing e-th roots modulo N). No efficient *classical* factoring algorithm is known; the best general method, the General Number Field Sieve, runs in sub-exponential but super-polynomial time $\exp\big(O(\log N)^{1/3}(\log\log N)^{2/3}\big)$.

Remark 3.5 (Textbook vs. real RSA). "Textbook RSA" as above is deterministic and therefore *insecure* in practice (equal messages give equal ciphertexts, small m with small e may satisfy $m^e < N$ so $c = m^e$ exactly, etc.). Real deployments use randomised padding (RSA-OAEP for encryption, RSA-PSS for signatures) and $e = 65537$. These do not change the number theory above; they protect against structural attacks.

4 Shor's algorithm

Shor's algorithm (1994) factors N in time polynomial in $\log N$ on a quantum computer, breaking RSA asymptotically. Its heart is a purely classical reduction (factoring → order finding) wrapped around one quantum subroutine (order finding via phase estimation).

4.1 Step 1 — Reduce factoring to order finding

Definition 4.1 (Multiplicative order)**.** For a with $\gcd(a, N) = 1$, the *order* $r = \mathrm{ord}_N(a)$ is the smallest positive integer with $a^r \equiv 1 \pmod N$.

Theorem 4.2 (Order finding yields factors)**.** *Let N be odd and composite, a coprime to N with order r. If*

*(i) r is **even**, and*

(ii) $a^{r/2} \not\equiv -1 \pmod N$ (note $a^{r/2} \not\equiv +1$ automatically, by minimality of r),

*then $\gcd\big(a^{r/2} - 1,\, N\big)$ and $\gcd\big(a^{r/2} + 1,\, N\big)$ are **non-trivial factors** of N.*

Proof. Set $x = a^{r/2} \bmod N$. Then $x^2 = a^r \equiv 1$, so

$$N \mid x^2 - 1 = (x-1)(x+1).$$

If N divided $x - 1$ we would have $x \equiv 1$, contradicting minimality of r; if N divided $x + 1$ we would have $x \equiv -1$, excluded by (ii). Hence N divides the product $(x-1)(x+1)$ but neither factor alone,

so its prime factors are split between the two. Therefore $\gcd(x-1, N)$ is a proper divisor > 1 of N (and so is $\gcd(x+1, N)$). □

Lemma 4.3 (The gamble usually pays off)**.** *If N has k distinct odd prime factors and a is drawn uniformly from $(\mathbb{Z}/n\mathbb{Z})^*$, then conditions (i)–(ii) of Theorem 4.2 both hold with probability at least $1 - 2^{-k} \geq \frac{1}{2}$.*

So on average two or three random choices of a suffice; if a fails, pick another.

4.2 Step 2 — Compute the order on a quantum computer

The only hard step is finding $r = \mathrm{ord}_N(a)$. Classically this is as hard as factoring; quantumly it is *easy*, via Quantum Phase Estimation (QPE) of the operator

$$U|y\rangle = |ay \bmod N\rangle, \qquad y \in \{0, 1, \ldots, N-1\}.$$

U is unitary (multiplication by a permutes the residues coprime to N).

Proposition 4.4 (Eigenstructure of U)**.** *For each $s \in \{0, 1, \ldots, r-1\}$ the state*

$$|u_s\rangle = \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} e^{-2\pi i\, sk/r}\, |a^k \bmod N\rangle$$

is an eigenvector of U with eigenvalue $e^{2\pi i\, s/r}$:

$$U|u_s\rangle = e^{2\pi i\, s/r}|u_s\rangle.$$

Moreover

$$\frac{1}{\sqrt{r}} \sum_{s=0}^{r-1} |u_s\rangle = |1\rangle.$$

Idea. Applying U shifts $a^k \mapsto a^{k+1}$ in the sum, which is equivalent to multiplying the whole state by $e^{2\pi i s/r}$ after re-indexing (the phases were chosen precisely so that this works). Summing $|u_s\rangle$ over s, the phase factors $\sum_s e^{-2\pi i sk/r}$ vanish for $k \not\equiv 0$ and reinforce for $k = 0$, leaving only $|a^0\rangle = |1\rangle$. □

The eigenphase of each $|u_s\rangle$ is $\phi_s = s/r$ — it *contains the order*. Two facts make this usable:

- We cannot prepare a single $|u_s\rangle$ (we don't know r), but we *can* prepare $|1\rangle = \frac{1}{\sqrt{r}} \sum_s |u_s\rangle$ trivially.
- Running QPE on U with input $|1\rangle$ therefore measures a phase $\phi \approx s/r$ for a **uniformly random** $s \in \{0, \ldots, r-1\}$.

The QPE circuit. With t *counting* qubits (initialised to $|+\rangle^{\otimes t}$) and a work register holding $|1\rangle$, apply controlled $U^{2^0}, U^{2^1}, \ldots, U^{2^{t-1}}$, then the inverse Quantum Fourier Transform on the counting register, then measure it. The read-out integer y estimates $\phi \approx y/2^t \approx s/r$. Using $t \approx 2\log_2 N + 1$ counting qubits makes the estimate sharp enough that continued fractions recover r with constant probability.

4.3 Step 3 — Recover r by continued fractions

QPE returns a binary fraction $\phi = y/2^t$ close to the rational s/r. We need s/r in lowest terms.

Theorem 4.5 (Continued-fraction recovery)**.** *If a rational s/r with $0 \le s < r \le N$ is approximated by ϕ to within $|\,\phi - s/r\,| \le \dfrac{1}{2r^2}$, then s/r is one of the* convergents *of the continued-fraction expansion of ϕ, and is found in $O(\log N)$ steps.*

In code this is a single call: `Fraction(phi).limit_denominator(N)` returns the best rational approximation to ϕ with denominator $\le N$ — exactly the convergent guaranteed above. The denominator of that fraction is the candidate for r.

Remark 4.6 (Why the algorithm is probabilistic). Three things can go wrong on a given run: the random s may be 0 (useless), s and r may share a factor (we then recover a *divisor* of r, not r itself), or the recovered r may be odd / give $a^{r/2} \equiv -1$. Each failure is detected cheaply ($a^r \overset{?}{\equiv} 1$, and the gcd test), and we simply repeat. A constant number of repetitions gives any desired success probability.

4.4 The complete recipe

Recipe S1 — Shor's factoring algorithm for N

1. **Classical shortcuts.** If N is even, output 2. If $N = c^b$ is a perfect power (checkable classically), output c.
2. Pick a random $a \in \{2, \dots, N-1\}$. Compute $g = \gcd(a, N)$. If $g > 1$, output g (a lucky factor).
3. **Quantum order finding.** Use QPE on $U|y\rangle = |ay \bmod N\rangle$ with input $|1\rangle$ to measure $\phi \approx s/r$.
4. **Continued fractions.** Set $r =$ denominator of `limit_denominator`$(\phi,\, N)$ (Theorem 4.5). Verify $a^r \equiv 1 \pmod N$; if not, go to step 2.
5. **Extract factors.** If r is even and $a^{r/2} \not\equiv -1 \pmod N$, output $\gcd(a^{r/2} \pm 1,\, N)$ (Theorem 4.2). Otherwise go to step 2.

4.5 Worked quantum examples

Example 4.7 (Factoring $N = 15$ with $a = 7$)**.** The order of 7 modulo 15 is $r = 4$ (since $7^1 = 7$, $7^2 = 49 \equiv 4$, $7^3 \equiv 13$, $7^4 \equiv 1$). QPE with $t = 8$ counting qubits returns one of the phases $\phi \in \{0, \frac{1}{4}, \frac{1}{2}, \frac{3}{4}\}$, i.e. register values $\{0, 64, 128, 192\}$ out of 256. Taking $\phi = \frac{3}{4}$: `Fraction(0.75).limit_denominator(15)`$= 3/4$, so $r = 4$. Then

$$a^{r/2} = 7^2 = 49 \equiv 4 \pmod{15}, \quad \gcd(4-1, 15) = 3, \quad \gcd(4+1, 15) = 5.$$

We recover $15 = 3 \times 5$. (The outcomes $\phi = 0$ and $\phi = \frac{1}{2}$ fail and are retried — this is exactly the histogram you see in the notebook.)

Example 4.8 (Factoring $N = 21$ with $a = 2$)**.** Powers of 2 mod 21: $2, 4, 8, 16, 11, 1$, so $r = 6$ (even). Then $a^{r/2} = 2^3 = 8$, and

$$\gcd(8-1, 21) = \gcd(7, 21) = 7, \qquad \gcd(8+1, 21) = \gcd(9, 21) = 3,$$

giving $21 = 3 \times 7$.

Example 4.9 (Breaking the toy RSA key of Example 3.2 by factoring)**.** Suppose an attacker sees only the public key $(N, e) = (55, 3)$ and the ciphertext $c = 13$. Running Recipe S1 on $N = 55$ recovers $p = 5, q = 11$. Then $\varphi(N) = 40$, $d \equiv 3^{-1} \equiv 27 \pmod{40}$, and $m = c^d \bmod N = 13^{27} \bmod 55 = 7$. The private key — and the message — are recovered purely from public data. This is the attack `16_Shor_Period_Finding_to_RSA.ipynb` performs on a simulator in its "breaking RSA" section.

4.6 The bottleneck: modular exponentiation

The controlled-U^{2^j} gates dominate the cost. We never build U^{2^j} by repeating U a full 2^j times (that is exponential); instead $a^{2^j} \bmod N$ is precomputed *classically* by repeated squaring in $O(j)$ multiplications, and the modular multiplier is realised as a reversible circuit. Beauregard's construction factors an n-bit N with only $2n + 3$ qubits and $O(n^3)$ gates.

Theorem 4.10 (Complexity of Shor's algorithm)**.** *Order finding, and hence factoring N, succeeds with constant probability using $O(\log N)$ qubits and $\widetilde{O}\big((\log N)^2\big)$ quantum gates per run (dominated by modular exponentiation), plus $O((\log N)^3)$ classical post-processing — polynomial in the input size $\log N$.*

4.7 Reality check and post-quantum cryptography

Although Shor's algorithm is polynomial, the *constants* and the cost of quantum error correction are enormous. The widely cited estimate of Gidney and Ekerå (2019) puts the cost of factoring a 2048-bit RSA modulus at roughly 20 million *noisy physical* qubits running for about 8 hours, corresponding to a few thousand logical (error-corrected) qubits. Today's devices — including IQM *Garnet* (20 qubits) and *Emerald* (54 qubits), and the largest superconducting chips (a few hundred physical qubits) — are many orders of magnitude short of this. Hence the companion notebooks factor 15 and 21, not real keys.

The threat is nevertheless taken seriously ("harvest now, decrypt later"), which is why NIST has standardised **post-quantum** schemes whose security does *not* reduce to factoring or discrete logs — notably the lattice-based ML-KEM (Kyber) and ML-DSA (Dilithium).

5 From the lesson to the notebook

The companion notebook `16_Shor_Period_Finding_to_RSA.ipynb` realises this lesson cell by cell:

- §2 ↔ `factors_from_order`: Theorem 4.2 in code.
- §4.2 ↔ `build_order_finding_circuit`, `c_amod15`, `qft_dagger`: the QPE circuit of Proposition 4.4.
- §4.3 ↔ `best_rational` / `limit_denominator`: continued-fraction recovery (Theorem 4.5).
- Recipe S1 ↔ `shor_factor(N)`: the full loop.
- Example 3.2 and the "breaking RSA" cell ↔ Theorem 3.1 and Proposition 3.4.

Run it on the Aer simulator (`IBM/16_Shor_Period_Finding_to_RSA.ipynb`) or on IQM Garnet/Emerald (`IQM/16_Shor_Period_Finding_to_RSA.ipynb`) by changing a single `TARGET` line.

Exercises

1. Compute $\varphi(561)$ and verify that $561 = 3 \cdot 11 \cdot 17$ is a Carmichael number ($a^{560} \equiv 1$ for all a coprime to 561, yet 561 is composite).
2. For $p = 7, q = 13$ build an RSA key with $e = 5$; find d and encrypt/decrypt $m = 9$.
3. Prove that if e is chosen with $\gcd(e, \varphi(N)) \neq 1$ then d need not exist, and explain why Recipe R1 step 3 forbids it.
4. In Example 4.7, which measured phases ϕ *fail* to produce a factor, and why? Relate your answer to the Remark on probabilistic failure.
5. Factor $N = 35$ by hand using $a = 3$ ($r = 12$): show $\gcd(3^6 \pm 1, 35)$ gives $\{5, 7\}$.

References

[1] R. Rivest, A. Shamir, L. Adleman, *A Method for Obtaining Digital Signatures and Public-Key Cryptosystems*, Comm. ACM **21** (1978) 120–126.

[2] P. W. Shor, *Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer*, SIAM J. Comput. **26** (1997) 1484; arXiv:quant-ph/9508027.

[3] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, Cambridge Univ. Press (2000), Ch. 5.

[4] S. Beauregard, *Circuit for Shor's algorithm using* $2n + 3$ *qubits*, arXiv:quant-ph/0205095.

[5] C. Gidney and M. Ekerå, *How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits*, Quantum **5** (2021) 433; arXiv:1905.09749.